Week 12 Graded Assignment Architecture Summary

Use Case: Ecommerce

The Graded project demonstrates an **Ecommerce multi-agent system** built using **CrewAI**.

CrewAI is one of the **top multi-agent frameworks** used for agent collaboration. It enables the creation of **teams of AI agents** that work together like a crew to achieve a shared goal. Instead of a single agent handling all tasks, multiple agents are assigned **specific roles**, allowing complex problems to be solved more efficiently.

CrewAI Components:

A CrewAI multi-agent system consists of **three main components**:

1. Agents: Agents are defined with a **specific role, goal, and backstory**. Each agent is responsible for performing a particular function within the system.

2. Tasks: Tasks define the **workflow and instructions** that agents must follow. Based on the task structure, agents execute their responsibilities and generate outputs.

3. Crew: The crew represents the **orchestration layer** where all agents are connected. Agent execution starts from the first agent, and the output of each agent is passed to the next agent until the final goal is achieved.

Architecture Diagram:

The Ecommerce support system is designed to **analyse and respond to customer queries** and provide appropriate solutions. The system first classifies the customer issue, checks it against company policies, generates a resolution, and escalates the issue to a human agent when necessary.

Agents Roles in the System

1. Issue_Classification_Agent

This agent analyzes the customer query and **classifies the type of issue** (e.g., order issue, payment issue, delivery issue).

2. Policy_Reasoning_Agent

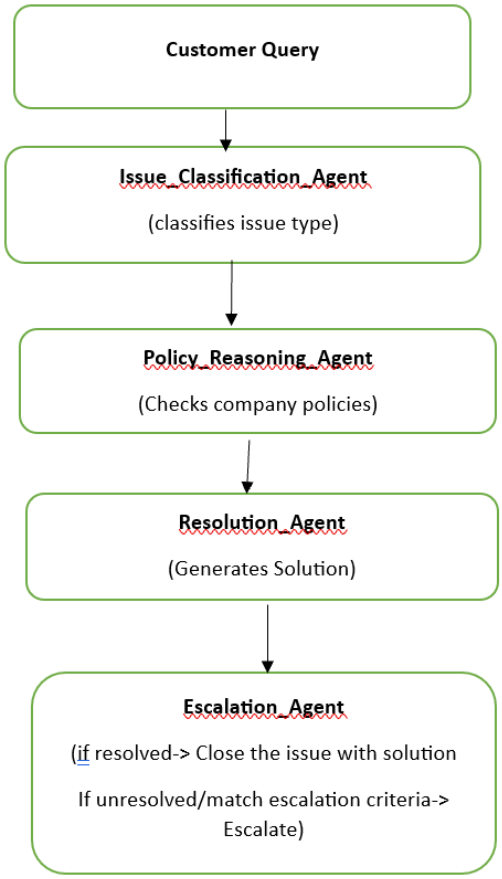
Based on the output from the **Issue_Classification_Agent**, this agent checks whether the issue matches any **company policies**.

3. Resolution_Agent

This agent receives inputs from both the **Issue_Classification_Agent** and the **Policy_Reasoning_Agent** and generates the **most appropriate resolution** for the customer issue.

4. Escalation_Agent

This agent evaluates the response provided by the **Resolution_Agent** and determines whether **human escalation** is required. If escalation is necessary, the issue is forwarded to a human support agent for further investigation.



Task Flow/ Handoffs:

The system consists of four tasks mapped to four agents.

identify_issue: (issue_classification_agent)
This task analyzes the customer query and determines the issue type such as refund, return, replacement, urgency, or sentiment. A structured summary is passed to the next agent.

fetch_policy: (Policy_Reasoning_agent)
This task receives the summary from the Issue_Classification_Agent and checks the knowledge base for relevant policy matches. If a match is found, it is returned; otherwise, a “No Match Found” response is generated.

generate_resolution: (Resolution_agent)
This task generates an empathetic and policy-aware response for the customer based on the outputs from previous agents. The agent focuses only on resolution generation and does not independently decide escalation.

apply_escalation: (Escalation agent)
This task performs rule-based checks to determine whether human escalation is required based on predefined conditions.

Sequential Handoff Pattern:

The system follows a sequential handoff pattern where agents execute in a fixed order: issue_classification, Policy_Reasoning, resolution and escalation.

The Escalation agent performs rule-based checks on the final output to determine whether escalation is required, but it does not alter the execution path making the overall flow sequential rather then conditional.

Escalation_logic:

The escalation_agent follows a rule based decision mechanism, where escalation decisions are made using predefined conditions such as policy availability, confidence score thresholds, urgency level, completeness of input details and whether human safety compromised.

Sample input/outputs:

Scenario 1:

```
customer_query = """
I ordered the Aqualogica Vitamin C serum from Amazon, but the bottle came broken.I want a replacement or
refund immediately. This is clearly a delivery issue.
"""
```

--- FINAL SYSTEM OUTPUT ---

{

"final_status": "resolved",

"reason": "A clear policy was found and applied, a resolution path (replacement or refund) has been identified, and the next steps are clearly outlined awaiting customer preference. None of the escalation criteria are met."

}

Scenario 2:

```
customer_query = """
i have received the an empty package when i ordered anarkali set from Biba. Dont know know what exactly
happened. it is not expected from such a reputed brand. Can you help me know what happened with my
package and send the product ASAP.
"""

result = Ecommerce_Crew.kickoff(
    inputs={"customer_query": customer_query}
)

print("\n--- FINAL SYSTEM OUTPUT ---\n")
print(result)
```

--- FINAL SYSTEM OUTPUT ---

{

"final_status": "resolved",

"reason": "Policy was found and applied, resolution path is clear and certain, and there is no indication of missing required details or low confidence."

}

Scenario 3:

```
customer_query = """
ordered 'paneer butter masala' from your restraunt and recieved 'Chicken butter masala'. Receiving a non
veg food being a veg family is disgusting that too on vaikunta ekadashi just because of your mistake.
need full refund ASAP and not ordering food from your restraunt again
"""
```

--- FINAL SYSTEM OUTPUT ---

{

"final_status": "resolved",

"reason": "A refund has been processed and confirmed, with no further action required from the customer. None of the escalation criteria (policy mismatch, low confidence, high urgency/uncertain resolution, missing details, previous agent escalation, or human safety compromise) are met."

}

Escalation Scenarios:

1. **Scenario 1**

```
customer_query = """
Ordered chicken pickle from X which is in India to be delevered in the US-Florida. Ordered the package on
14th january and its 9th February still not recieved the package. can you please let me know what
happened and send the package
"""
```

--- FINAL SYSTEM OUTPUT ---

{

"final_status": "requires_escalation",

"reason": "The previous agent has indicated that the situation is unique and requires a thorough review by a specialized policy team to determine the clear path forward, which constitutes an internal escalation."

}

Scenario 2:

```
customer_query = """
i have ordered BP tablets from Apollo Pharmacy online and received the expired one and my mother is
unwell now and admitted in hospital. How can people be such irresponsible on things that cost a life. Who
will be responsible if anything happened to my mother. I want to take an action on this can somebody help?
"""
```

--- FINAL SYSTEM OUTPUT ---

```json
{
  "final_status": "requires_escalation",
  "reason": "The automated system could not identify a standard policy or action, and the previous agent/system has already escalated the case to a specialized team for manual review."
}
```

Scenario 3:

```
customer_query = """
Driver misbehavior when used the UBER cab service to commute from office to home. Luckily escaped from
the situation as the police arrived. Dont know how you compromise girls safety by hiring some stupid
people. i need an answer from your manager, what if something happened to me?
"""
```

--- FINAL SYSTEM OUTPUT ---

```
{
  "final_status": "requires_escalation",
  "reason": "The automated system could not identify a standard policy or action to resolve the issue, and the case has already been escalated to a specialized team for a thorough manual review, fulfilling the 'Policy is NO MATCH FOUND' and 'previous agent says escalation' criteria for human intervention."
}
```

Design Rationale:

I chose this architecture because it closely replicates the real-world issue resolution flow followed by ecommerce and on-demand services. Ecommerce platforms such as product delivery, food delivery, home services, and ride-hailing encounter different types of customer issues. This architecture is designed to handle these variations effectively. In a typical ecommerce customer support workflow, the issue is first identified, then validated against company policies, and finally resolved with an appropriate solution or escalation. My architecture mirrors this process through sequential agents for issue classification, policy reasoning, resolution, and escalation. This design ensures clarity, consistency, and realistic handling of customer issues.